claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a0158bc680310a701.jsonl`
- SHA-256: `58a3e56a7ea0264131a9085b22030ed5ff882d96f506a31ac7665cd23f0354fe`
- Source modified: `2026-06-10T16:31:35+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:28:11.318Z)

```
You are an adversarial verifier. A code reviewer audited the repo at
C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Cr/Dr suffix sign-stripped in parse_amount ? an overdraft (Dr-balance) statement
reconciles with every debit/credit INVERTED and persists as verified
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/validation.py (line 32, 45)
DESCRIPTION: parse_amount strips a trailing 'Cr'/'Dr' and returns the magnitude with no sign:
'1,500.00 Dr' -> +1500.0. For an account whose balance is negative throughout (OD facility / EMI
overdue ? Dr-suffixed balances are how Indian banks print these), the magnitude sequence of a
negative-balance walk is exactly a sign-flipped positive walk, so the L2 mapping search
(engine.py:285-293) finds that assigning the Withdrawals column to 'credit' and Deposits to
'debit' makes the arithmetic work, and map_table returns it verified=True. Empirically confirmed
on an AXIS_PROFILE header-less matrix (exactly what cluster_to_matrix emits, since
txn_line_predicate drops header lines): true events 'withdraw 500, deposit 300, withdraw 800' on
balances -1000->-1500->-1200->-2000 printed as '1,500.00 Dr' etc. came back `verified: True,
layer: L2, map {'debit': 3, 'credit': 2}` ? every direction inverted, balances stored positive.
The total-reconcile bookend does NOT save this even if finding #1 is fixed:
parse_statement_summary's regex also captures magnitudes only, and magnitudes are self-
consistent under inversion. This is the worst-case class: wrong rows that still reconcile.
EVIDENCE QUOTED: validation.py:32: `_CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")`; :45:
`text = _CR_DR_SUFFIX.sub("", str(value).strip())...` (sign discarded). engine.py:43: `_MONEY_RE
= re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")`. Empirical: map_table on Dr-suffixed
balances returned `verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 3,
'credit': 2, 'balance': 4}` with 'UPI OUT' stored as credit=500.0.
SUGGESTED FIX: Make parse_amount sign-aware: a 'Dr' suffix on a balance yields a negative value
(Cr positive), or return (magnitude, suffix) and have the engine reject candidate mappings that
contradict the printed suffix. Additionally, have map_table detect AMBIGUITY: if more than one
candidate mapping passes the oracle (the inverted and correct mappings both pass for sign-
symmetric ledgers), refuse to auto-select and quarantine.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario
can actually occur in realistic single-user use on Windows, and whether the assigned severity is
justified.
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot
confirm the problem is real, set isReal=false. If it is real but over/under-stated, set
adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL
financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file
existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess
sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers,
```

balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.

2. user (2026-06-10T16:28:16.655Z)

```
1       """The **verifier** ? reconciliation as a layout-independent truth oracle.
2
3       Reconciliation proves a parse is **complete and correct without inspecting any sensitive
value**:
4       if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5       parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6       money ? *extraction proposes, reconciliation disposes* (see
7       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §2, §5).
8
9       This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(works even
13        with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16        endorses (the seam the L2 mapping search plugs into).
17      * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
18        assumption.
19
20      Reports carry counts / booleans / a verdict only ? **never an amount**.
21      """
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
27      from dataclasses import dataclass
28      from enum import StrEnum
29
30      from .concepts import ConceptRow, StatementConcepts
31
32      _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35      def parse_amount(value: object) -> float | None:
36          """Parse a statement money cell to a float.
37
38          Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"`` /
39          non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
40          *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
41          ``None`` and would corrupt a real zero balance and the reconciliation math.
42          """
43          if value is None:
44              return None
45          text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46          if text in ("", "-"):
47              return None
48          try:
```

```
            return float(text)
        except ValueError:
            return None


    @dataclass
    class ReconcileReport:
        """Result of a running-balance walk. Counts only, never an amount."""

        n: int  # rows considered
        compared: int  # consecutive (balance-bearing) pairs actually checked
        breaks: int
        first_break: int | None = None

        @property
        def ok(self) -> bool:
            return self.compared > 0 and self.breaks == 0


    def reconcile_running_balance(
        entries: Sequence[tuple[float | None, float | None, float | None]],
        tol: float = 0.01,
    ) -> ReconcileReport:
        """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.

        ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
        ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
        is skipped (can't anchor on it). Because the walk spans the whole statement, a
        clean result also rules out any row missed *between* two balances ? which is the
        failure mode silent ``extract_tables()`` row-drops would otherwise hide.
        """
        breaks = 0
        first_break: int | None = None
        compared = 0
        prev: float | None = None
        for i, (debit, credit, balance) in enumerate(entries):
            if balance is None:
                continue
            if prev is not None:
                expected = prev - (debit or 0.0) + (credit or 0.0)
                compared += 1
                if abs(expected - balance) > tol:
                    breaks += 1
                    if first_break is None:
                        first_break = i
            prev = balance
        return ReconcileReport(
            n=len(entries), compared=compared, breaks=breaks, first_break=first_break
        )


    def _row_entries(
        rows: Sequence[ConceptRow],
    ) -> list[tuple[float | None, float | None, float | None]]:
        """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
consumes."""
        return [(r.debit, r.credit, r.balance) for r in rows]


    def reconcile_total(
        rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
    ) -> bool:
        """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.

        This is the oracle for statements with **no reliable per-row balance** (e.g. Axis
```

```
savings):
        it checks the whole statement against its own opening/closing bookends without
needing each
        row's running balance. ``opening`` and ``closing`` come from the bank's
SUMMARY/header
        (``StatementConcepts``); the caller must have them.
        """
        delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
        return abs((opening + delta) - closing) <= tol


    class Confidence(StrEnum):
        """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).

        ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
        a check failed (or none could run); the parse is suspect and is quarantined rather
than trusted.
        """

        RECONCILED = "reconciled"
        UNRECONCILED = "unreconciled"


    @dataclass
    class BookendReport:
        """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.

        ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
        requires the per-row balance walk *plus* every available check to pass.
        """

        n: int
        reconciles: bool
        debit_count_ok: bool | None = None
        credit_count_ok: bool | None = None
        opening_ok: bool | None = None
        closing_ok: bool | None = None

        @property
        def ok(self) -> bool:
            checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
            return self.n > 0 and self.reconciles and all(c is True for c in checks)

        @property
        def confidence(self) -> Confidence:
            return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED


    def verify(
        rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
    ) -> BookendReport:
        """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.

        The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
        extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
        figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
        implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
```

```
opening).
        """
        reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
        debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
        if concepts.debit_count is not None:
            debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
        if concepts.credit_count is not None:
            credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
        if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
            closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
        if rows and rows[0].balance is not None and concepts.opening_balance is not None:
            implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
            opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
        return BookendReport(
            n=len(rows),
            reconciles=reconciles,
            debit_count_ok=debit_count_ok,
            credit_count_ok=credit_count_ok,
            opening_ok=opening_ok,
            closing_ok=closing_ok,
        )


    def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
        """Among candidate concept-row mappings, return the index of the first that
reconciles.

        This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
        extraction ladder can't be sure which column?concept assignment is right, it
proposes several
        and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.

        The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
        the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
        running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
        """
        for i, rows in enumerate(candidates):
            if reconcile_running_balance(_row_entries(rows), tol).ok:
                return i
        return None


    __all__ = [
        "parse_amount",
        "ReconcileReport",
        "reconcile_running_balance",
        "reconcile_total",
        "Confidence",
        "BookendReport",
        "verify",
        "select_reconciling",
    ]

```

3. user (2026-06-10T16:28:17.480Z)

src\muneem\parsers__init__.py

src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\engine.py
src\muneem\parsers\base.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py

4. user (2026-06-10T16:28:26.103Z)

```
1       """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3       This is the *generic-first* half of the concept-core architecture (see
4       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §4, §6).
5       Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
6       or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7       via a cheap-first ladder, with a profile supplying priors:
8
9       * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
10      * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
11      * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
12        plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
13        reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
14        makes the math work* rather than by its label.
15
16      The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
17      names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
18      (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
19      the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20      value; provenance is the winning layer + column map.
21      """
22
23      from __future__ import annotations
24
25      import itertools
26      import re
27      from collections.abc import Callable, Sequence
28      from dataclasses import dataclass, field
29
30      from .concepts import ConceptRow
31      from .validation import parse_amount
32
33      Cell = object  # a table cell: str | None (we coerce defensively)
34      Row = Sequence[Cell]
35
36      # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
37      _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
38          re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"),  # dd/mm/yyyy or dd-mm-yyyy
39          re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"),  # dd-Mon-yyyy / dd Mon yy
40      )
41      # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
```

```
    # for an amount column during content classification.
    _MONEY_RE = re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")
    _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")

    # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names, values are
    # lower-cased header labels matched exactly (not by substring) during L0.
    _BANK_ALIASES: dict[str, tuple[str, ...]] = {
        "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
        "value_date": ("value date", "val date", "value dt"),
        "narration": (
            "narration",
            "particulars",
            "description",
            "transaction details",
            "details",
            "remarks",
            "transaction",
            "transaction remarks",
        ),
        "reference": (
            "ref",
            "reference",
            "ref no",
            "reference no",
            "cheque no",
            "chq no",
            "instrument no",
        ),
        "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
        "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
        "balance": (
            "balance",
            "closing balance",
            "running balance",
            "closing bal",
            "available balance",
        ),
    }


    def _text(cell: Cell) -> str:
        return "" if cell is None else str(cell).strip()


    def _norm(label: str) -> str:
        return re.sub(r"\s+", " ", label.strip().lower())


    def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
        return any(p.match(text) for p in patterns)


    def _looks_like_money(text: str) -> bool:
        return bool(_MONEY_RE.match(text))


    @dataclass
    class FamilySpec:
        """A concept *family*: the target vocabulary + how to build typed rows from a column mapping.

        The engine's L0/L1/L2 logic is written against this spec rather than against bank concepts, so a
        future instrument family (with its own concepts + reconciliation oracle) reuses the
```

```
same engine.
104         ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105         """
106
107         name: str
108         concepts: tuple[str, ...]
109         date_concepts: tuple[str, ...]  # in order: the first is the primary (row-gating)
date
110         amount_concepts: tuple[str, ...]  # money columns; the non-balance ones are the
"flows"
111         balance_concept: str | None  # the running-balance concept, if the family has one
112         text_concepts: tuple[str, ...]  # free-text columns; the first gets the widest text
column
113         default_aliases: dict[str, tuple[str, ...]]
114         build: Callable[[list[dict[str, str]]], list]
115
116
117     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
118         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134
135     BANK_FAMILY = FamilySpec(
136         name="bank",
137         concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
138         date_concepts=("txn_date", "value_date"),
139         amount_concepts=("debit", "credit", "balance"),
140         balance_concept="balance",
141         text_concepts=("narration", "reference"),
142         default_aliases=_BANK_ALIASES,
143         build=_build_bank_rows,
144     )
145
146
147     @dataclass
148     class MappingResult:
149         """The outcome of mapping one table: typed rows + how we got there (provenance)."""
150
151         rows: list  # typed rows from family.build (e.g. list[ConceptRow])
152         mapped: list[
153             dict[str, str]
154         ]  # per-row {concept: raw cell}, incl. any profile-declared extra columns
155         column_map: dict[str, int]  # the winning concept -> column-index assignment
156         layer: str  # "L0" / "profile-order" / "L2" ? which rung produced it
157         verified: bool  # did the oracle endorse it? (False if no oracle was supplied)
158
159
160     @dataclass
161     class Profile:
```

```
        """Declarative per-issuer knowledge ? *data, not code* (architecture §6).

        Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
        accepted date shape, a positional fallback when there's no header, table anchors,
account-number
        patterns, quirks, and (placeholder, wiring parked) the password rule.
        """

        name: str
        aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
        date_patterns: tuple[re.Pattern, ...] = ()
        column_order: tuple[str, ...] | None = None
        table_anchors: tuple[str, ...] = ()
        account_patterns: tuple[re.Pattern, ...] = ()
        quirks: frozenset[str] = frozenset()
        password_rule: str | None = None  # data only; sourcing is parked (DESIGN §8.2)

        def first_account(self, text: str) -> str | None:
            for rx in self.account_patterns:
                m = rx.search(text)
                if m:
                    return m.group(1).strip()
            return None


    def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
        """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
        index: dict[str, str] = {}
        for concept, labels in family.default_aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        for concept, labels in profile.aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        return index


    def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
        return profile.date_patterns or _GENERIC_DATE_PATTERNS


    def _detect_header(
        rows: list[list[Cell]], alias_index: dict[str, str]
    ) -> tuple[int | None, dict[str, int]]:
        """Find the header row (most exact alias hits, >=2) and its concept->column map."""
        best_idx: int | None = None
        best_map: dict[str, int] = {}
        for i, row in enumerate(rows):
            cmap: dict[str, int] = {}
            for col, cell in enumerate(row):
                concept = alias_index.get(_norm(_text(cell)))
                if concept and concept not in cmap:
                    cmap[concept] = col
            if len(cmap) > len(best_map):
                best_map, best_idx = cmap, i
        if len(best_map) < 2:
            return None, {}
        return best_idx, best_map


    def _is_balance_marker(row: Row) -> bool:
        return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)

```

```
224
225     def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
226         if date_col is not None:
227             cell = row[date_col] if date_col < len(row) else None
228             return _matches_date(_text(cell), patterns)
229         return any(_matches_date(_text(c), patterns) for c in row)
230
231
232     def _classify_columns(
233         data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
234     ) -> dict[int, str]:
235         """Tag each column date / amount / text / empty by majority content."""
236         kinds: dict[int, str] = {}
237         for c in range(ncols):
238             cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
239             if not cells:
240                 kinds[c] = "empty"
241                 continue
242             thresh = max(1, int(len(cells) * 0.6))
243             if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
244                 kinds[c] = "date"
245             elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
246                 kinds[c] = "amount"
247             else:
248                 kinds[c] = "text"
249         return kinds
250
251
252     def _base_map(
253         col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
254     ) -> dict[str, int]:
255         """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.
256
257         Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
258         goes to the family's first text concept (if any). Amount columns are left to the
search.
259         """
260         base: dict[str, int] = {}
261         date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
262         text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
263         for concept, col in zip(family.date_concepts, date_cols, strict=False):
264             base[concept] = col
265         if text_cols and family.text_concepts:
266             base[family.text_concepts[0]] = max(
267                 text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
268             )
269         return base
270
271
272     def _search_candidates(
273         base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
274     ) -> list[tuple[str, dict[str, int]]]:
275         """Enumerate plausible amount-column ? concept assignments (the L2 search space).
276
277         Family-agnostic: the "flows" are the family's amount concepts minus its balance
concept (for
278         banks: debit, credit). We try each amount column as the balance and assign the rest
to the
279         flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
Oct).
280         """
281         amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
```

```
        flows = [c for c in family.amount_concepts if c != family.balance_concept]
        bal = family.balance_concept
        cands: list[dict[str, int]] = []
        if bal and len(amount_cols) >= len(flows) + 1:
            for bcol in amount_cols:
                rest = [c for c in amount_cols if c != bcol]
                for assign in itertools.permutations(rest, len(flows)):
                    cands.append({**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
        if flows and len(amount_cols) >= len(flows):
            for assign in itertools.permutations(amount_cols, len(flows)):
                cands.append({**base, **dict(zip(flows, assign, strict=True))})
        return [("L2", c) for c in cands[:24]]


    def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
        out: list[dict[str, str]] = []
        for row in data:
            out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
        return out


    def map_table(
        table: Sequence[Row] | None,  # Sequence (covariant) so list[list[str | None]]
callers fit
        profile: Profile,
        family: FamilySpec = BANK_FAMILY,
        *,
        oracle: Callable[[list], bool] | None = None,
    ) -> MappingResult | None:
        """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.

        Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
        ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
        structurally-complete candidate is returned unverified (the caller reconciles
separately).
        Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
        was supplied but nothing reconciled.
        """
        if not table:
            return None
        rows_in = [list(r) for r in table if r is not None]
        if not rows_in:
            return None
        alias_index = _alias_index(profile, family)
        patterns = _date_patterns(profile)

        header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
        start = header_idx + 1 if header_idx is not None else 0
        date_col_hint = col_map_hdr.get("txn_date")
        if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
            date_col_hint = profile.column_order.index("txn_date")

        data = [
            r
            for r in rows_in[start:]
            if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
        ]
        if not data:
```

```
338             return None
339         ncols = max(len(r) for r in data)
340         col_kinds = _classify_columns(data, ncols, patterns)
341         base = _base_map(col_kinds, data, family)
342         flows = [c for c in family.amount_concepts if c != family.balance_concept]
343 
344         candidates: list[tuple[str, dict[str, int]]] = []
345         if flows and all(c in col_map_hdr for c in flows):  # L0: header pinned the flow
concepts
346             candidates.append(("L0", {**base, **col_map_hdr}))
347         elif col_map_hdr:  # header present but didn't pin the flows: keep its hints, search
the rest
348             candidates.append(("L0", {**base, **col_map_hdr}))
349         if profile.column_order:  # positional prior (no/partial header)
350             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351         candidates += _search_candidates(base, col_kinds, family)
352 
353         primary_date = family.date_concepts[0] if family.date_concepts else None
354         best: MappingResult | None = None
355         for layer, cmap in candidates:
356             if primary_date and primary_date not in cmap:
357                 continue
358             if not (set(family.amount_concepts) & cmap.keys()):
359                 continue
360             mapped = _apply_map(data, cmap)
361             typed = family.build(mapped)
362             if oracle is None:
363                 return MappingResult(typed, mapped, cmap, layer, verified=False)
364             if oracle(typed):
365                 return MappingResult(typed, mapped, cmap, layer, verified=True)
366             if best is None:
367                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
368         return best
369 
370 
371     __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372 
```

5. user (2026-06-10T16:28:36.235Z)

```
1       """Axis Bank statement parsers: savings/transaction account + credit card.
2 
3       **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4       coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5       vintage it can't read stores nothing (``unreconciled``) rather than mislabeled rows.
Axis specifics
6       live entirely in ``AXIS_PROFILE``; this replaced the old fixed-column ``extract_tables``
mapping.
7 
8       Credit-card statements carry no running balance, so ``AxisCreditCardParser`` can't
reconcile
9       (parse/persist split + count only). No statement content is logged here.
10      """
11 
12      from __future__ import annotations
13 
14      import re
15      import sqlite3
16      from dataclasses import dataclass
17      from pathlib import Path
18 
19      import pdfplumber
```

```
20
21      from muneem import __version__
22      from muneem import db as dbmod
23      from muneem.errors import ParseError
24      from muneem.redaction import redact_filename
25
26      from . import register_parser
27      from .base import StatementParser
28      from .profiles import AXIS_PROFILE
29      from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30      from .validation import Confidence, parse_amount
31
32      _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}")  # Axis credit card: dd/mm/yyyy
33      _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*+\d{4})")]
34
35
36      def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37          for rx in patterns:
38              m = rx.search(text)
39              if m:
40                  return m.group(1).strip()
41          return None
42
43
44      # --- Savings / transaction account (concept core)
----------------------------------------------
45
46
47      # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48      AxisSavings = SavingsParse
49      parse_axis_summary = parse_statement_summary
50      _is_txn_line = txn_line_predicate(AXIS_PROFILE)
51
52
53      def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54          """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55          return parse_savings(pdf_path, AXIS_PROFILE, password)
56
57
58      @register_parser
59      class AxisParser(StatementParser):
60          def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61              if sender and "axisbank.com" in sender.lower():
62                  return True
63              t = text or ""
64              return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66          def parse_and_insert(
67              self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68          ) -> int:
69              """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71              Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72              parser can't yet read surfaces loudly instead of saving mislabeled rows.
73              """
74              try:
75                  parsed = parse_axis_savings(pdf_path, password)
76              except ParseError:
```

```
                raise
            except Exception as exc:
                # Filename only ? never statement text; original cause kept via `from`.
                name = redact_filename(pdf_path)
                raise ParseError(f"failed to parse Axis statement {name}") from exc

            if not parsed.verified:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AxisParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_axis_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AxisParser",
                parser_version=__version__,
            )
            return len(parsed.rows)


    # --- Credit card (no running balance; count only)
-------------------------------------------


    @dataclass
    class AxisCCTxn:
        txn_date: str
        particulars: str
        merchant_category: str
        amount: float
        type: str  # "Cr" or "Dr"


    def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
        """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
        rows: list[AxisCCTxn] = []
        for table in tables or []:
            for row in table or []:
                if not row or len(row) < 9:
                    continue
                date_str = str(row[0]).strip() if row[0] else ""
                if not _CC_DATE_RE.match(date_str):
                    continue
                amount_str = str(row[8]).strip() if row[8] else ""
                is_cr = "Cr" in amount_str
                amount = parse_amount(re.sub(r"[^\d.]", "", amount_str)) or 0.0
                rows.append(
```

```
                    AxisCCTxn(
                        txn_date=date_str,
                        particulars=str(row[2]).strip() if row[2] else "",
                        merchant_category=str(row[7]).strip() if row[7] else "",
                        amount=amount,
                        type="Cr" if is_cr else "Dr",
                    )
                )
        return rows


    def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
        """Open an Axis credit-card statement and return (card_number, transactions)."""
        card_number = "UNKNOWN"
        rows: list[AxisCCTxn] = []
        with pdfplumber.open(pdf_path, password=password) as pdf:
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
            card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
            for page in pdf.pages:
                rows.extend(_rows_from_cc_tables(page.extract_tables()))
        return card_number, rows


    @register_parser
    class AxisCreditCardParser(StatementParser):
        def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
            return "Axis Bank" in text and "Credit Card Statement" in text

        def parse_and_insert(
            self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
        ) -> int:
            try:
                card_number, rows = parse_axis_cc(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(
                    f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
                ) from exc
            for t in rows:
                dbmod.insert_axis_cc_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=card_number,
                    txn_date=t.txn_date,
                    particulars=t.particulars,
                    merchant_category=t.merchant_category,
                    amount=t.amount,
                    type=t.type,
                )
            # No running balance to reconcile; documents.status stays as the caller set it.
We still
            # record provenance (with no confidence verdict).
            dbmod.record_provenance(
                conn,
                document_id=doc_id,
                source_type="pdf_statement",
                parser="AxisCreditCardParser",
                parser_version=__version__,
                mapping_layer="deterministic",
            )
```

```
201             return len(rows)
202
```

6. user (2026-06-10T16:28:36.573Z)

```
1       """Coordinate clustering ? turn positioned words into a clean cell-matrix.
2
3       When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned
statement (the
4       Axis case: a date merged into the narration cell, column counts drifting between
vintages), we read
5       **word positions** directly. Words are grouped into visual rows by their ``top``; column
boundaries
6       are inferred from the vertical whitespace **shared across the transaction rows**; and
each word is
7       dropped into the column its x-centre falls in. Because columns come from geometry, a
leading date
8       sits in its *own* column instead of being glued to the narration.
9
10      Two robustness lessons baked in:
11
12      * **Infer columns from transaction-like rows only.** Headers, address blocks, and
summaries span the
13        page and would erase the inter-column whitespace. A caller-supplied ``keep_line``
predicate (e.g.
14        "has a date *and* a money token") restricts both column inference and the emitted
matrix to the
15        transaction band.
16      * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
empty
17        in *most* rows; a single long narration can't collapse two columns.
18
19      Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's
oracle still
20      decides whether the result is trustworthy. Pure geometry in, strings out ? no value is
logged.
21      """
22
23      from __future__ import annotations
24
25      from collections.abc import Callable, Sequence
26
27      Word = dict  # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
28
29
30      def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
31          """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
32          lines: list[list[Word]] = []
33          anchor_top: float | None = None
34          for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
35              if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
36                  lines[-1].append(w)
37              else:
38                  lines.append([w])
39                  anchor_top = w["top"]
40          return [sorted(line, key=lambda w: w["x0"]) for line in lines]
41
42
43      def infer_columns(
44          lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
45      ) -> list[tuple[float, float]]:
46          """Infer column x-extents from a coverage histogram over ``lines``.
47
48          An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
```

```
columns are
49          the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
50          to a few full-width lines: it thresholds on the *fraction* of rows, not on any
single one.
51          """
52          words = [w for line in lines for w in line]
53          if not words:
54              return []
55          x0 = min(float(w["x0"]) for w in words)
56          x1 = max(float(w["x1"]) for w in words)
57          nbins = max(1, int((x1 - x0) / bin_w) + 1)
58          cover = [0] * nbins
59          for line in lines:
60              marked: set[int] = set()
61              for w in line:
62                  a = int((float(w["x0"]) - x0) / bin_w)
63                  b = int((float(w["x1"]) - x0) / bin_w)
64                  marked.update(range(max(0, a), min(nbins - 1, b) + 1))
65              for k in marked:
66                  cover[k] += 1
67          thresh = max(1.0, min_frac * len(lines))
68          filled = [c >= thresh for c in cover]
69          cols: list[tuple[float, float]] = []
70          start: int | None = None
71          for i, f in enumerate(filled):
72              if f and start is None:
73                  start = i
74              elif not f and start is not None:
75                  cols.append((x0 + start * bin_w, x0 + i * bin_w))
76                  start = None
77          if start is not None:
78              cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
79          return cols
80
81
82      def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
83          for i, (a, b) in enumerate(columns):
84              if a <= center <= b:
85                  return i
86          return min(
87              range(len(columns)),
88              key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
89          )
90
91
92      def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) ->
list[str]:
93          """Place each word into its column; join words that share a column (in x order)."""
94          cells = ["" for _ in columns]
95          for w in line:
96              idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
97              cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
98          return cells
99
100
101     def cluster_to_matrix(
102         words_by_page: Sequence[Sequence[Word]],
103         *,
104         line_tol: float = 2.5,
105         min_frac: float = 0.3,
106         keep_line: Callable[[list[str]], bool] | None = None,
107     ) -> list[list[str]]:
108         """Cluster words across pages into one cell-matrix with consistent columns.
109
```

```
110         ``keep_line`` (texts of a line ? bool) restricts both column inference and the
output to the
111         transaction band ? without it, full-width headers/footers swamp the column
structure. Columns
112         are inferred once across all kept lines (a statement keeps its layout); rows are
emitted in
113         page-then-top order.
114         """
115         lines: list[list[Word]] = []
116         for page in words_by_page:
117             lines.extend(group_lines(page, line_tol))
118         if keep_line is not None:
119             lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
120         if not lines:
121             return []
122         columns = infer_columns(lines, min_frac=min_frac)
123         if not columns:
124             return []
125         return [line_to_cells(line, columns) for line in lines]
126
127
128     __all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]
129
```

7. user (2026-06-10T16:28:45.309Z)

```
1       """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3       Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4       and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6       1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7          coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8       2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9       3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10         vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12      A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13      on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14      cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15      can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.
16
17      No statement content is logged; only counts/booleans travel out.
18      """
19
20      from __future__ import annotations
21
22      import re
23      from dataclasses import dataclass, field
24      from pathlib import Path
25
26      import pdfplumber
27
28      from .clustering import cluster_to_matrix
29      from .concepts import ConceptRow, StatementConcepts
30      from .engine import Profile, map_table
```

```
    from .validation import parse_amount, reconcile_running_balance, reconcile_total

    _MONEY_TOKEN = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
    _OPENING_RE = re.compile(r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
    _CLOSING_RE = re.compile(r"closing\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)


    @dataclass
    class SavingsParse:
        """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""

        account_number: str
        rows: list[ConceptRow] = field(default_factory=list)
        verified: bool = False


    def parse_statement_summary(text: str) -> StatementConcepts:
        """Read opening/closing balance from statement text (for the total-reconcile
oracle).

        Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
        """
        opening = _OPENING_RE.search(text)
        closing = _CLOSING_RE.search(text)
        return StatementConcepts(
            opening_balance=parse_amount(opening.group(1)) if opening else None,
            closing_balance=parse_amount(closing.group(1)) if closing else None,
        )


    def txn_line_predicate(profile: Profile):
        """A line is transaction-like if it carries one of the profile's dates *and* a money
token.

        Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
        address blocks that would otherwise wreck column inference).
        """
        patterns = profile.date_patterns

        def keep(texts: list[str]) -> bool:
            has_date = any(any(p.match(t) for p in patterns) for t in texts)
            return has_date and any(_MONEY_TOKEN.match(t) for t in texts)

        return keep


    def reconcile_oracle(opening: float | None, closing: float | None):
        """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""

        def check(rows: list[ConceptRow]) -> bool:
            if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
                return True
            if rows and opening is not None and closing is not None:
                return reconcile_total(rows, opening, closing)
            return False

        return check


    def _reconstruct_by_balance_delta(
        matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
```

```
90      ) -> list[ConceptRow]:
91          """Recover debit/credit from the running balance when columns won't split cleanly.
92
93          Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
94          it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
95          single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
96          caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
97          a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance
98          to fix the first row's direction; returns ``[]`` if it can't form a clean series.
99          """
100         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
101         rows = [[(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
102         if not rows or opening is None:
103             return []
104         ncols = max(len(r) for r in rows)
105
106         def is_date(s: str) -> bool:
107             return any(p.match(s) for p in patterns)
108
109         def money(s: str) -> float | None:
110             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
111
112         # date column = the leftmost of the (first few) columns with the most date cells
113         date_counts: dict[int, int] = {}
114         for r in rows:
115             for c in range(min(ncols, 3)):
116                 if c < len(r) and is_date(r[c]):
117                     date_counts[c] = date_counts.get(c, 0) + 1
118         if not date_counts:
119             return []
120         top = max(date_counts.values())
121         date_col = min(c for c, n in date_counts.items() if n == top)
122
123         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
124         if len(txn_rows) < 2:
125             return []
126
127         # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
128         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
129         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
130         money_cols = []
131         for c in range(ncols):
132             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
133             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
134             if m >= 1 and m > d:
135                 money_cols.append(c)
136         if len(money_cols) < 2:
137             return []
138         balance_col = max(money_cols)
139         flow_cols = [c for c in money_cols if c != balance_col]
140
141         text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
142         narr_col = (
143             max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
144             if text_cols
145             else None
146         )
```

```

        out: list[ConceptRow] = []
        prev = opening
        for r in txn_rows:
            bal = money(r[balance_col]) if balance_col < len(r) else None
            if bal is None:
                return []  # a running balance on every row is required for the delta
            amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
            debit = credit = None
            if amt is not None:
                if bal < prev - tol:
                    debit = amt
                elif bal > prev + tol:
                    credit = amt
                else:
                    debit = amt  # zero net with an amount present ? reconciliation will
flag it
            out.append(
                ConceptRow(
                    txn_date=r[date_col],
                    narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
                    debit=debit,
                    credit=credit,
                    balance=bal,
                )
            )
            prev = bal
        return out


    def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
        """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.

        Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
        and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
        reconciles, so the caller can refuse to persist.
        """
        with pdfplumber.open(pdf_path, password=password) as pdf:
            words_by_page = [
                page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
            ]
            tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = profile.first_account(head) or "UNKNOWN"
        summary = parse_statement_summary(head)
        oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

        confirmed: list[ConceptRow] = []
        for table in tables:
            res = map_table(table, profile, oracle=oracle)
            if res and res.verified:
                confirmed.extend(res.rows)
        if confirmed and oracle(confirmed):
            return SavingsParse(account_number, confirmed, True)

        matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
```

```
203         clustered = map_table(matrix, profile, oracle=oracle)
204         if clustered and clustered.verified:
205             return SavingsParse(account_number, clustered.rows, True)
206
207         # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
208         #    debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
209         #    direction from the running balance and let the oracle cross-check amount ==
|?balance|.
210         delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
211         if delta and oracle(delta):
212             return SavingsParse(account_number, delta, True)
213
214         fallback = confirmed or (clustered.rows if clustered else [])
215         return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225
```

8. user (2026-06-10T16:28:45.525Z)

```
1       """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
§6).
2
3       A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4       concept?label aliases, the accepted date shape, a positional fallback when there's no
header,
5       table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
an issuer is
6       ideally *adding a profile*, not writing a parser.
7
8       `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
coordinate-
9       clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
table (see the
10      `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
engine-driven
11      parsers (coordinate clustering ? generic mapping ? total reconcile).
12      """
13
14      from __future__ import annotations
15
16      import re
17
18      from .engine import Profile
19
20      AXIS_PROFILE = Profile(
21          name="axis",
22          aliases={
23              # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
| Balance |
24              # Other Information. "Transaction" is the narration column (not the date).
25              "narration": ("transaction", "particulars"),
26              "debit": ("withdrawals", "withdrawal", "debit"),
27              "credit": ("deposits", "deposit", "credit"),
28              "balance": ("balance",),
29          },
```

```
30          date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),),  # dd-mm-yyyy or
dd/mm/yyyy
31          # Layout / column-count varies between vintages, so there is NO reliable
column_order ? the
32          # generic content search + total reconcile do the work. Broadened account pattern:
the old
33          # `Account No:` regex missed real statements.
34          account_patterns=(
35              re.compile(r"Account\s*(?:No|Number)\s*[:.\-]?\s*(\d{6,})", re.IGNORECASE),
36              re.compile(r"A/c\s*No\.?\s*[:\s]*(\d{6,})", re.IGNORECASE),
37          ),
38          quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39      )
40
41      AUBANK_PROFILE = Profile(
42          name="aubank",
43          aliases={
44              "narration": (
45                  "transaction",
46                  "particulars",
47                  "description",
48                  "narration",
49                  "transaction details",
50              ),
51              "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52              "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53              "balance": ("balance", "closing balance", "running balance"),
54          },
55          # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56          date_patterns=(
57              re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58              re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59          ),
60          account_patterns=(
61              re.compile(r"Account\s*(?:No|Number)\s*[:.\-]?\s*(\d{6,})", re.IGNORECASE),
62              re.compile(r"A/c\s*No\.?\s*[:\s]*(\d{6,})", re.IGNORECASE),
63          ),
64          quirks=frozenset({"engine-driven"}),
65      )
66
67      HDFC_PROFILE = Profile(
68          name="hdfc",
69          aliases={
70              "debit": ("withdrawal", "withdrawals"),
71              "credit": ("deposit", "deposits"),
72              "balance": ("closing balance",),
73          },
74          date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),),  # dd/mm/yyyy
75          table_anchors=("Narration", "Closing Balance"),
76          account_patterns=(re.compile(r"Account\s*Number\s*[:.]?\s*(\d{6,})",
re.IGNORECASE),),
77          # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78          # by word-position clustering rather than through the generic cell-matrix engine.
79          quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80      )
81
82      __all__ = ["AXIS_PROFILE", "AUBANK_PROFILE", "HDFC_PROFILE"]
83
```

9. user (2026-06-10T16:29:31.161Z)

```
tests\parsers\test_axis.py:121:            ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food",
"250.00 Dr"],
```

```
tests\parsers\test_axis.py:122:            ["06/02/2026", "x", "Refund", "", "", "", "", "Food",
"100.00 Cr"],
tests\parsers\test_axis.py:128:    assert rows[0].amount == 250.0 and rows[0].type == "Dr" and
rows[0].merchant_category == "Food"
tests\parsers\test_axis.py:129:    assert rows[1].amount == 100.0 and rows[1].type == "Cr"
tests\parsers\test_hdfc.py:221:    HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and
reconcile fully.
tests\parsers\test_hdfc.py:236:        f"dr={report.debit_count_ok} cr={report.credit_count_ok}
"
```

10. user (2026-06-10T16:29:32.700Z)

```
src\muneem\parsers\hdfc.py:45:    BookendReport,
src\muneem\parsers\hdfc.py:109:ValidationReport = BookendReport
src\muneem\parsers\hdfc.py:364:    return verify(rows, summary, tol)
src\muneem\parsers\validation.py:15:* **selection** ? ``select_reconciling``: among candidate
concept mappings, pick the one arithmetic
src\muneem\parsers\validation.py:17:* **confidence** ? ``Confidence`` +
``BookendReport.confidence``: a recorded verdict, not an
src\muneem\parsers\validation.py:133:class BookendReport:
src\muneem\parsers\validation.py:157:def verify(
src\muneem\parsers\validation.py:159:) -> BookendReport:
src\muneem\parsers\validation.py:178:    return BookendReport(
src\muneem\parsers\validation.py:188:def select_reconciling(candidates:
Sequence[Sequence[ConceptRow]], tol: float = 0.01) -> int | None:
src\muneem\parsers\validation.py:211:    "BookendReport",
src\muneem\parsers\validation.py:213:    "select_reconciling",
```

11. user (2026-06-10T16:29:46.662Z)

```
.claude
.githooks
.github
.mypy_cache
.pytest_cache
.ruff_cache
.venv
docs
src
testdata
tests
tools
.coverage
.gitattributes
.gitignore
CLAUDE.md
DESIGN.md
Makefile
pyproject.toml
README.md
venv exists
```

12. assistant (2026-06-10T16:30:04.826Z)

```
"""Adversarial repro: Dr-suffixed (negative) balance walk vs map_table. Synthetic data only."""

from muneem.parsers.engine import map_table
from muneem.parsers.profiles import AXIS_PROFILE
from muneem.parsers.savings import parse_statement_summary, reconcile_oracle
from muneem.parsers.validation import parse_amount
```

1. parse_amount sign behaviour

```
print("parse_amount('1,500.00 Dr') =", parse_amount("1,500.00 Dr"))
print("parse_amount('1,500.00 Cr') =", parse_amount("1,500.00 Cr"))
```

2. Header-less matrix (what cluster_to_matrix emits): true events on an OD account open -1000; withdraw 500 -> -1500; deposit 300 -> -1200; withdraw 800 -> -2000. Printed Indian-style: magnitude + ' Dr'. Columns: date | narration | deposits | withdra

```
matrix = [
    ["01-02-2026", "UPI OUT", "", "500.00", "1,500.00 Dr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "1,200.00 Dr"],
    ["03-02-2026", "UPI OUT", "", "800.00", "2,000.00 Dr"],
]
summary_text = "Opening Balance 1,000.00 Dr\nClosing Balance 2,000.00 Dr"
summary = parse_statement_summary(summary_text)
print("summary opening/closing =", summary.opening_balance, summary.closing_balance)
oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

res = map_table(matrix, AXIS_PROFILE, oracle=oracle)
print("verified:", res.verified, "layer:", res.layer, "map:", res.column_map)
for r in res.rows:
    print("  row:", r.txn_date, repr(r.narration), "debit=", r.debit, "credit=", r.credit,
"balance=", r.balance)
```

3. Same but WITH a header row (extract_tables path) ? does L0 failing fall through to i

```
table_with_header = [
    ["Tran Date", "Transaction", "Deposits", "Withdrawals", "Balance"],
    ["01-02-2026", "UPI OUT", "", "500.00", "1,500.00 Dr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "1,200.00 Dr"],
    ["03-02-2026", "UPI OUT", "", "800.00", "2,000.00 Dr"],
]
res2 = map_table(table_with_header, AXIS_PROFILE, oracle=oracle)
print("with header -> verified:", res2.verified, "layer:", res2.layer, "map:", res2.column_map)
```

4. Control: positive (Cr-suffixed) walk must map correctly.

```
matrix_cr = [
    ["01-02-2026", "UPI OUT", "", "500.00", "500.00 Cr"],
    ["02-02-2026", "NEFT IN", "300.00", "", "800.00 Cr"],
]
oracle_cr = reconcile_oracle(1000.0, 800.0)
res3 = map_table(matrix_cr, AXIS_PROFILE, oracle=oracle_cr)
print("Cr control -> verified:", res3.verified, "layer:", res3.layer, "map:", res3.column_map)
```

5. Mixed-sign walk (crosses zero): should fail both mappings -> quarantined (fail-safe?

```
matrix_mixed = [
    ["01-02-2026", "UPI OUT", "", "500.00", "500.00 Cr"],
    ["02-02-2026", "UPI OUT", "", "800.00", "300.00 Dr"],
]
oracle_mx = reconcile_oracle(1000.0, -300.0)
res4 = map_table(matrix_mixed, AXIS_PROFILE, oracle=oracle_mx)
print("mixed-sign -> verified:", res4.verified if res4 else None, "layer:", res4.layer if res4
else None)
```

13. user (2026-06-10T16:30:04.837Z)

```
File created successfully at: C:\Users\avidu\Projects\muneem\tools\_verify_drcr_repro.py (file
state is current in your context ? no need to Read it back)
```

14. user (2026-06-10T16:30:13.656Z)

```
parse_amount('1,500.00 Dr') = 1500.0
parse_amount('1,500.00 Cr') = 1500.0
summary opening/closing = 1000.0 2000.0
verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 2, 'credit': 3,
'balance': 4}
  row: 01-02-2026 'UPI OUT' debit= None credit= 500.0 balance= 1500.0
```

```
  row: 02-02-2026 'NEFT IN' debit= 300.0 credit= None balance= 1200.0
  row: 03-02-2026 'UPI OUT' debit= None credit= 800.0 balance= 2000.0
with header -> verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 2,
'credit': 3, 'balance': 4}
Cr control -> verified: True layer: L2 map: {'txn_date': 0, 'narration': 1, 'debit': 3,
'credit': 2, 'balance': 4}
mixed-sign -> verified: False layer: L2
```

15. assistant (2026-06-10T16:30:50.387Z)

```
The repro confirms the finding exactly ? and slightly worse than claimed. Let me clean up the
temp script.
```

16. user (2026-06-10T16:30:53.123Z)

```
 M docs/feedbackJune2026.md
?? docs/feedback-antigravity-response.md
```

17. user (2026-06-10T16:31:27.491Z)

```
Structured output provided successfully
```

18. assistant (2026-06-10T16:31:35.424Z)

```
Verification complete.
```